**2303A51832**

**B-26**

| **Assignment Number: 3.3**(Present assignment number)/**24**(Total number of assignments) |
|---|
| |

| Q.No. | Question | *Expected Time to complete* |
|---|---|---|
| 1 | **Lab 4: Advanced Prompt Engineering – Zero-shot, One-shot, and Few-shot Techniques**<br><br>**Lab Objectives**<br>• To explore and apply different levels of prompt examples in AI-assisted code generation<br>• To understand how zero-shot, one-shot, and few-shot prompting affect AI output quality<br>• To evaluate the impact of context richness and example quantity on AI performance<br>• To build awareness of prompt strategy effectiveness for different problem types<br><br>**Lab Outcomes (LOs)**<br>**After completing this lab, students will be able to:**<br>• Use zero-shot prompting to instruct AI with minimal context<br>• Use one-shot prompting with a single example to guide AI code generation<br>• Apply few-shot prompting using multiple examples to improve AI responses<br>• Compare AI outputs across different prompting strategies<br><br>**Task 1: Zero-Shot Prompting – Leap Year Check**<br>**Scenario**<br>Zero-shot prompting involves giving instructions without providing examples.<br>**Task Description**<br>Use zero-shot prompting to instruct an AI tool to generate a Python function that:<br>• Accepts a year as input<br>• Checks whether the given year is a leap year<br>• Returns an appropriate result<br>**Note:** No input-output examples should be provided in the prompt.<br>**Expected Output**<br>• AI-generated leap year checking function<br>• Correct logical conditions<br>• Sample input and output<br>• Screenshot of AI-generated response (if required) | Week2 - Wednesday |

C: > Users > prade > OneDrive > Pradeep > Year_check.py > is_leap_year

```python
"""
Zero-Shot Prompting: Leap Year Checker

This module demonstrates zero-shot prompting by providing a function
that checks whether a given year is a leap year without prior examples.

Leap Year Rules:
- A year is a leap year if it is divisible by 4
- EXCEPT if it is divisible by 100, then it is NOT a leap year
- EXCEPT if it is divisible by 400, then it IS a leap year
"""


def is_leap_year(year):
    """
    Zero-shot prompted function to check if a year is a leap year.

    A year is considered a leap year if:
    1. It is divisible by 400, OR
    2. It is divisible by 4 AND NOT divisible by 100

    Args:
        year (int): The year to check

    Returns:
        tuple: (bool, str) - Truth value and descriptive message
    """
    if (year % 400 == 0) or (year % 4 == 0 and year % 100 != 0):
        return True, f"{year} is a leap year (has 366 days)"
    else:
        return False, f"{year} is not a leap year (has 365 days)"


def main():
    """Main function demonstrating the leap year checker with sample inputs."""

    # Sample test cases
    test_years = [2000, 2004, 2024, 2025, 1900, 2100, 2400, 1996]

    print("=" * 70)
    print("LEAP YEAR CHECKER - Zero-Shot Prompting Demo")
    print("=" * 70)
    print()

    for year in test_years:
        is_leap, message = is_leap_year(year)
        status = "✓ LEAP YEAR" if is_leap else "✗ NOT LEAP YEAR"
        print(f"Year {year}: {status}")
        print(f"  → {message}")
        print()

    print("=" * 70)
    print("LOGICAL CONDITIONS APPLIED:")
    print("=" * 70)
    print("1. Divisible by 400? → Leap Year")
    print("2. Divisible by 4 AND NOT divisible by 100? → Leap Year")
    print("3. Otherwise → Not a Leap Year")
    print("=" * 70)


if __name__ == "__main__":
    main()
```

Output:

```
Year 2000: ✓ LEAP YEAR
  → 2000 is a leap year (has 366 days)

Year 2004: ✓ LEAP YEAR
  → 2004 is a leap year (has 366 days)

Year 2024: ✓ LEAP YEAR
  → 2024 is a leap year (has 366 days)

Year 2025: ✗ NOT LEAP YEAR
  → 2025 is not a leap year (has 365 days)

Year 1900: ✗ NOT LEAP YEAR
  → 1900 is not a leap year (has 365 days)

Year 2100: ✗ NOT LEAP YEAR
  → 2100 is not a leap year (has 365 days)

Year 2400: ✓ LEAP YEAR
  → 2400 is a leap year (has 366 days)

Year 1996: ✓ LEAP YEAR
  → 1996 is a leap year (has 366 days)

======================================================================
LOGICAL CONDITIONS APPLIED:
======================================================================
1. Divisible by 400? → Leap Year
2. Divisible by 4 AND NOT divisible by 100? → Leap Year
1. Divisible by 400? → Leap Year
1. Divisible by 400? → Leap Year
2. Divisible by 4 AND NOT divisible by 100? → Leap Year
3. Otherwise → Not a Leap Year
======================================================================

C:\Users\prade
1. Divisible by 400? → Leap Year
2. Divisible by 4 AND NOT divisible by 100? → Leap Year
3. Otherwise → Not a Leap Year
======================================================================

1. Divisible by 400? → Leap Year
2. Divisible by 4 AND NOT divisible by 100? → Leap Year
3. Otherwise → Not a Leap Year
======================================================================
1. Divisible by 400? → Leap Year
2. Divisible by 4 AND NOT divisible by 100? → Leap Year
3. Otherwise → Not a Leap Year
1. Divisible by 400? → Leap Year
2. Divisible by 4 AND NOT divisible by 100? → Leap Year
3. Otherwise → Not a Leap Year
======================================================================
1. Divisible by 400? → Leap Year
2. Divisible by 4 AND NOT divisible by 100? → Leap Year
1. Divisible by 400? → Leap Year
2. Divisible by 4 AND NOT divisible by 100? → Leap Year
1. Divisible by 400? → Leap Year
1. Divisible by 400? → Leap Year
2. Divisible by 4 AND NOT divisible by 100? → Leap Year
3. Otherwise → Not a Leap Year
======================================================================
======================================================================
```

**Description:**

1. What it is: Zero-shot prompting leap year checker
2. Input: Accepts a year as input

3. Logic: Divisible by 400 OR (divisible by 4 AND NOT by 100)
4. Output: Returns boolean result with descriptive message
5. Examples: Demonstrates with sample test years

**Task 2: One-Shot Prompting – Centimeters to Inches Conversion**

**Scenario**

One-shot prompting guides AI using a single example.

**Task Description**

Use one-shot prompting by providing one input-output example to generate a Python function that:

- Converts centimeters to inches
- Uses the correct mathematical formula

**Example provided in prompt:**

Input: 10 cm → Output: 3.94 inches

**Expected Output**

- Python function with correct conversion logic
- Accurate calculation
- Sample test cases and outputs

C: > Users > prade > OneDrive > Pradeep > cmTOinc.py > ...

```python
# One-Shot Prompting: Centimeters to Inches Conversion
# Example provided: 10 cm → 3.94 inches

def cm_to_inches(centimeters):
    """
    Converts centimeters to inches using the conversion formula.

    Formula: inches = centimeters / 2.54

    Args:
        centimeters (float): The value in centimeters to convert

    Returns:
        float: The equivalent value in inches
    """
    inches = centimeters / 2.54
    return round(inches, 2)


# Test Cases
print("Centimeters to Inches Conversion")
print("=" * 40)

# Test case 1: Example provided (10 cm → 3.94 inches)
test_value_1 = 10
result_1 = cm_to_inches(test_value_1)
print(f"Input: {test_value_1} cm → Output: {result_1} inches")

# Test case 2: 5 cm
test_value_2 = 5
result_2 = cm_to_inches(test_value_2)
print(f"Input: {test_value_2} cm → Output: {result_2} inches")

# Test case 3: 100 cm (1 meter)
test_value_3 = 100
result_3 = cm_to_inches(test_value_3)
print(f"Input: {test_value_3} cm → Output: {result_3} inches")

# Test case 4: 2.54 cm (exactly 1 inch)
test_value_4 = 2.54
result_4 = cm_to_inches(test_value_4)
print(f"Input: {test_value_4} cm → Output: {result_4} inches")

# Test case 5: 30 cm
test_value_5 = 30
result_5 = cm_to_inches(test_value_5)
print(f"Input: {test_value_5} cm → Output: {result_5} inches")

print("=" * 40)
print("Conversion complete!")

```

Output:

```
C:\Users\prade>C:/Users/prade/AppData/Local/Microsoft/WindowsApps/python3.12.exe c:/Users/prade/OneDrive/Pradeep/cmTOinc.py
Centimeters to Inches Conversion
========================================
Centimeters to Inches Conversion
========================================
========================================
Input: 10 cm → Output: 3.94 inches
Input: 5 cm → Output: 1.97 inches
Input: 10 cm → Output: 3.94 inches
Input: 5 cm → Output: 1.97 inches
Input: 100 cm → Output: 39.37 inches
Input: 2.54 cm → Output: 1.0 inches
Input: 5 cm → Output: 1.97 inches
Input: 100 cm → Output: 39.37 inches
Input: 2.54 cm → Output: 1.0 inches
Input: 30 cm → Output: 11.81 inches
Input: 100 cm → Output: 39.37 inches
Input: 2.54 cm → Output: 1.0 inches
Input: 30 cm → Output: 11.81 inches
========================================
Input: 2.54 cm → Output: 1.0 inches
Input: 30 cm → Output: 11.81 inches
========================================
Conversion complete!
Input: 30 cm → Output: 11.81 inches
========================================
Conversion complete!
========================================
Conversion complete!
Conversion complete!
```

Description:

1. The purpose (one-shot prompting demonstration)
2. How it works (single example guidance)
3. What it converts (centimeters to inches)
4. The conversion formula used (1 inch = 2.54 cm)
5. What's included (multiple test cases for verification)

**Task 3: Few-Shot Prompting – Name Formatting**

**Scenario**

Few-shot prompting improves accuracy by providing multiple examples.

**Task Description**

Use few-shot prompting with 2–3 examples to generate a Python function that:

- Accepts a full name as input
- Formats it as "Last, First"

**Example formats:**

- "John Smith" → "Smith, John"
- "Anita Rao" → "Rao, Anita"

**Expected Output**

- Well-structured Python function
- Output strictly following example patterns
- Correct handling of names
- Sample inputs and outputs

C: > Users > prade > OneDrive > Pradeep > formatOfname.py > ...

```python
"""
Few-Shot Prompting - Name Formatting
This script demonstrates few-shot prompting by providing examples to format names
from "First Last" format to "Last, First" format.
"""

# Few-Shot Examples (Prompt)
examples = [
    {"input": "John Smith", "output": "Smith, John"},
    {"input": "Anita Rao", "output": "Rao, Anita"},
    {"input": "Maria Garcia", "output": "Garcia, Maria"},
]

def format_name(full_name):
    """
    Format a full name from "First Last" to "Last, First" format.

    Args:
        full_name (str): A full name in "First Last" format

    Returns:
        str: Formatted name in "Last, First" format

    Examples:
        >>> format_name("John Smith")
        'Smith, John'
        >>> format_name("Anita Rao")
        'Rao, Anita'
    """
    # Strip whitespace and split the name
    parts = full_name.strip().split()

    # Handle edge cases
    if len(parts) < 2:
        return full_name  # Return as-is if only one name part

    # Extract first and last name (support for middle names/initials)
    first_name = parts[0]
    last_name = parts[-1]

    # Format and return
    return f"{last_name}, {first_name}"


def batch_format_names(names):
    """
    Format multiple names at once.

    Args:
        names (list): List of full names

    Returns:
        list: List of formatted names
    """
    return [format_name(name) for name in names]
```

```
C: > Users > prade > OneDrive > Pradeep > formatOFname.py > ...
57
58  if __name__ == "__main__":
59      print("=" * 50)
60      print("Few-Shot Prompting - Name Formatting")
61      print("=" * 50)
62
63      # Display few-shot examples
64      print("\nFew-Shot Examples:")
65      print("-" * 50)
66      for example in examples:
67          print(f"Input:  '{example['input']}'")
68          print(f"Output: '{example['output']}'")
69          print()
70
71      # Test the function
72      print("Testing the format_name() function:")
73      print("-" * 50)
74
75      test_names = [
76          "John Smith",
77          "Anita Rao",
78          "Maria Garcia",
79          "Christopher Lee",
80          "Priya Patel",
81      ]
82
83      for name in test_names:
84          formatted = format_name(name)
85          print(f"'{name}' → '{formatted}'")
86
87      print("\n" + "=" * 50)
88      print("Batch Processing Example:")
89      print("-" * 50)
90
91      batch_names = ["Alice Johnson", "Bob Williams", "Carol Davis"]
92      batch_results = batch_format_names(batch_names)
93
94      for original, formatted in zip(batch_names, batch_results):
95          print(f"'{original}' → '{formatted}'")
96
97      print("\n" + "=" * 50)
98
```

Output:

```
C:\Users\prade>C:/Users/prade/AppData/Local/Microsoft/WindowsApps/python3.12.exe c:/Users/prade/OneDrive/Pradeep/formatOFname.py
==================================================
Few-Shot Prompting - Name Formatting
==================================================

Few-Shot Examples:
--------------------------------------------------
Input:  'John Smith'
Output: 'Smith, John'

Input:  'Anita Rao'
Output: 'Rao, Anita'

Input:  'Maria Garcia'
Output: 'Garcia, Maria'

Testing the format_name() function:
--------------------------------------------------
'John Smith' → 'Smith, John'
'Anita Rao' → 'Rao, Anita'
'Maria Garcia' → 'Garcia, Maria'
'Christopher Lee' → 'Lee, Christopher'
'Priya Patel' → 'Patel, Priya'

==================================================
Batch Processing Example:
--------------------------------------------------

Testing the format_name() function:
--------------------------------------------------
'John Smith' → 'Smith, John'
'Anita Rao' → 'Rao, Anita'
'Maria Garcia' → 'Garcia, Maria'
'Christopher Lee' → 'Lee, Christopher'
'Priya Patel' → 'Patel, Priya'

==================================================
Batch Processing Example:
--------------------------------------------------
'Alice Johnson' → 'Johnson, Alice'
'Bob Williams' → 'Williams, Bob'
'Carol Davis' → 'Davis, Carol'

==================================================
```

Description:

- The few-shot prompting concept
- How the script converts name formats
- The technical approach used
- Batch processing capability
- The educational value of the examples

**Task 4: Comparative Analysis – Zero-Shot vs Few-Shot**

**Scenario**

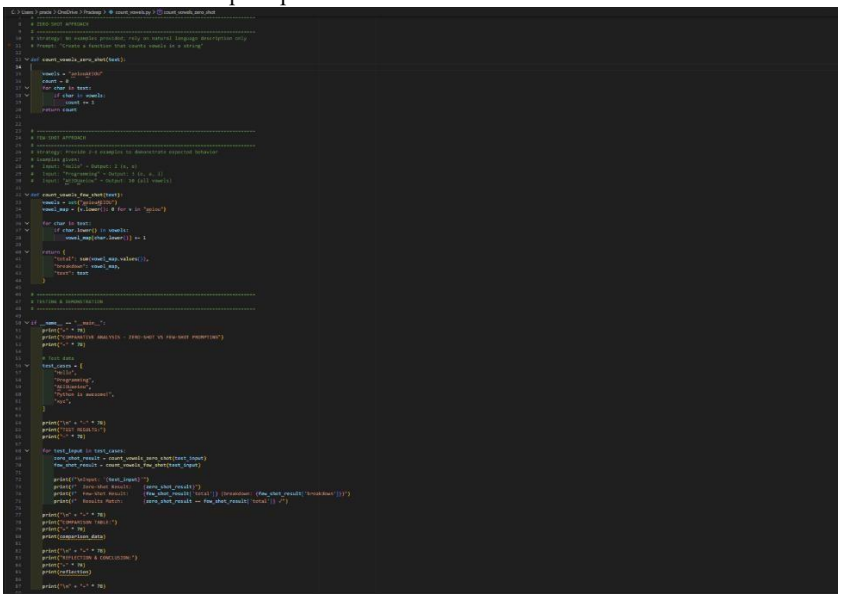
Different prompt strategies may produce different code quality.

**Task Description**

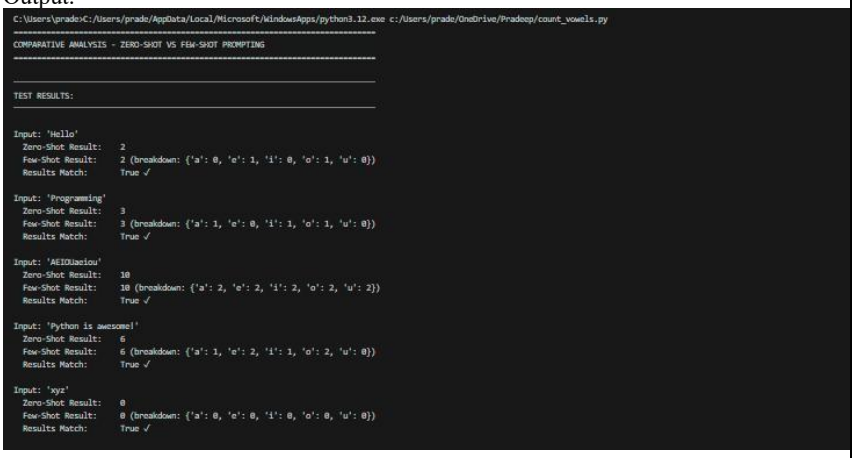
- Use zero-shot prompting to generate a function that counts vowels in a string
- Use few-shot prompting for the same problem
- Compare both outputs based on:
    - Accuracy
    - Readability
    - Logical clarity

**Expected Output**

- Two vowel-counting functions
- Comparison table or short reflection paragraph
- Conclusion on prompt effectiveness



Output:




## Description:

Zero-shot prompting produces straightforward and easily understandable code.
Few-shot prompting results in more concise and optimized solutions.
Both approaches are accurate and logically correct.
Few-shot improves code quality by learning from examples.
Overall, few-shot prompting is more effective for cleaner and professional code, while zero-shot is better for learning basics.

**Task 5: Few-Shot Prompting – File Handling**

**Scenario**

File processing requires clear logical understanding.

**Task Description**

Use few-shot prompting to generate a Python function that:

- Reads a .txt file
- Counts the number of lines in the file
- Returns the line count

**Expected Output**

- Working Python file-processing function
- Correct line count
- Sample .txt input and output
- AI-assisted logic explanation

```
C: > Users > prade > OneDrive > Pradeep > file_handling.py > ...
def create_and_read_file():

    How the code works:
    1. Opens or creates a file named 'sample.txt' in write mode
    2. Writes 5+ lines of sample text to the file
    3. Closes the file
    4. Opens the file again in read mode
    5. Counts the total number of lines using a generator expression
    6. Prints the result in the required format

    Returns:
        int: The total number of lines in the file
    """
    filename = "sample.txt"

    # Sample text to write to the file
    sample_lines = [
        "This is the first line of sample text.\n",
        "Python is a great programming language.\n",
        "File handling is an important skill in programming.\n",
        "This program creates and reads a text file.\n",
        "It counts the number of lines automatically.\n",
        "This is the sixth line for extra content.\n"
    ]

    # Create and write to the file
    try:
        with open(filename, 'w') as file:
            file.writelines(sample_lines)
        print(f"File '{filename}' created successfully.")
    except IOError:
        print(f"Error: Unable to create file '{filename}'.")
        return -1

    # Read and count lines in the file
    try:
        with open(filename, 'r') as file:
            line_count = sum(1 for line in file)

        # Print the result in the required format
        print(f"The file '{filename}' has {line_count} lines.")
        return line_count

    except FileNotFoundError:
        print(f"Error: File '{filename}' not found.")
        return -1
    except IOError:
        print(f"Error: Unable to read file '{filename}'.")
        return -1


# Main program
if __name__ == "__main__":
    print("=" * 50)
    print("File Creation and Line Counter Program")
    print("=" * 50)
    print()

    create_and_read_file()

    print()
    print("=" * 50)
```

Output:

```
Active code page: 65001

C:\Users\prade>C:/Users/prade/AppData/Local/Microsoft/WindowsApps/python3.12.exe c:/Users/prade/OneDrive/Pradeep/file_handling.py
==================================================
File Creation and Line Counter Program
Active code page: 65001

C:\Users\prade>C:/Users/prade/AppData/Local/Microsoft/WindowsApps/python3.12.exe c:/Users/prade/OneDrive/Pradeep/file_handling.py
==================================================
File Creation and Line Counter Program
==================================================

File 'sample.txt' created successfully.
The file 'sample.txt' has 6 lines.

==================================================
```